MAFIS: A Fault Resilience Observatory for Lifelong Multi-Agent Path Finding

Teddy Truong

Université du Québec à Chicoutimi (UQAC) / École Supérieure de Génie Informatique (ESGI)

Live demo · github.com/stasis-industries/mafis

Abstract. We present MAFIS, a browser-based observatory for measuring fault resilience in lifelong multi-agent path finding (MAPF). Existing MAPF tools evaluate solver performance under clean operating conditions; no tool measures how lifelong solvers degrade, recover, and adapt when robots fail permanently, zones become impassable, or mechanical wear accumulates. MAFIS fills this gap with a three-category fault taxonomy, twelve differential resilience metrics computed against a deterministic baseline, and real-time 3D visualization — all running in-browser via WebAssembly with shareable URLs. Five lifelong solvers are implemented from the literature. To demonstrate the observatory's utility, we report an 8,480-run experiment (Benjamini-Hochberg corrected, $\alpha = 0.05$) showing that fault-induced congestion relief is solver- and topology-dependent: burst failures improve throughput up to 2.1× for reactive solvers on corridor-based layouts but not on chokepoint layouts, while Token Passing collapses to 18% throughput under permanent zone failure (Cliff's $\delta = -1.0$). These findings are undetectable without MAFIS's differential measurement infrastructure. MAFIS is open-source (MIT), 30K lines of Rust, and compiles to a 15 MB browser binary.

1. Introduction

Multi-agent path finding (MAPF) research has focused on collision-free planning under clean conditions [9, 12, 14]. When robots crash, zones become inaccessible, or wear accumulates, the system must degrade gracefully. This recovery behavior is a first-class concern in deployed warehouse automation, yet no existing tool measures it systematically.

Prior work on execution robustness handles *delays*: temporary slowdowns absorbed with buffering [6], k-robust plans [2], or planning failure policies [11]. The League of Robot Runners [7] evaluates lifelong solvers under normal conditions and explicitly identifies fault tolerance as future work. Recent work on traffic flow optimization [5] documents throughput saturation at high density. None of these inject permanent failures or measure differential solver degradation.

MAFIS fills this gap. It is an empirical instrument — not a solver benchmark — that observes, measures, and visualizes how lifelong MAPF systems degrade under configurable fault injection. Its contributions:

(1) A three-category fault taxonomy (recoverable, permanent-distributed, permanent-localized) with Weibull-calibrated wear models (§3).
(2) Twelve differential resilience metrics computed against a paired fault-free baseline (§4).
(3) A browser-based 3D observatory with deterministic replay, cascade visualization, and shareable URLs — no installation required (§2, §7).
(4) A demonstration experiment revealing solver- and topology-dependent fault-induced congestion relief, with Benjamini-Hochberg corrected significance tests and Cliff's delta effect sizes (§6).

2. System Design

MAFIS is 30,000 lines of Rust using Bevy 0.18 ECS, compiled to WebAssembly for browser execution.

2.1. Deterministic ECS

The simulation runs in Bevy's `FixedUpdate` schedule at a fixed timestep, independent of frame rate. All random events use a single `SeededRng`; given the same seed, every simulation produces identical output. This enables *paired comparison*: the fault-free baseline runs with the same seed as the faulted simulation, so metric differences are causally attributable to the fault.

2.2. Bevy↔JS Bridge

A `thread_local` `RefCell` buffer bridges ECS and browser. A Bevy system serializes state to JSON at an adaptive interval (90 ms for $\leq$ 50 agents, 500 ms for 500) via `wasm-bindgen`. Commands flow in reverse: JS writes to the bridge, Bevy drains each frame. Above 50 agents, summaries replace per-agent snapshots.

2.3. Shareable URLs

Configuration serializes to JSON, compresses via Deflate (pako), and encodes as a URL fragment. No server required — the simulation is fully reconstructible from the URL alone.

3. Fault Taxonomy

MAFIS organizes faults into three categories by permanence and spatial extent (Table 1).

Category	Scenario	Effect
Recoverable	Zone outage	Latency on busiest zone for d ticks
	Intermittent	Exponential inter-arrival, 15-tick recovery
Perm.-distributed	Burst failure	Kill $k\%$ of fleet at tick t
	Wear-based	Weibull $F(t) = 1 - e^{\{-(t/\eta)^\beta\}}$ per agent
Perm.-localized	Perm. zone outage	Zone cells $\longrightarrow$ permanent obstacles at tick t

Table 1: Three-category fault taxonomy with five scenario types.

Wear presets are calibrated to published AGV reliability: Low ($\beta = 2.0, \eta = 900$, MTTF $\approx$ 798 ticks — CASUN AGV), Medium ($\beta = 2.5, \eta = 500$, MTTF $\approx$ 444 — Canadian AGV survey), High ($\beta = 3.5, \eta = 150$, MTTF $\approx$ 137 — Carlson & Murphy 2006).

A `FaultList` compiles multiple entries into a unified schedule; wear and permanent-zone faults are limited to one per simulation to prevent trivial collapse.

4. Resilience Metrics

All metrics are *differential*: comparing the faulted run to a fault-free baseline with the same seed. This paired design eliminates initialization variance.

Metric	Definition
Fault Tolerance	$P_{\text{fault}}/P_{\text{nominal}}$ — throughput retained vs. baseline
Throughput Recovery	Ticks until per-tick throughput returns to baseline rate
Deficit Recovery	Ticks until cumulative task deficit closes
NRR	$1 - \text{MTTR} / \text{MTBF}$ — normalized recovery ratio [13]
Critical Time	Fraction of post-fault ticks below 50% baseline throughput
Survival Rate	Alive agents / initial fleet at end
Propagation Rate	Avg. fraction of fleet affected per fault event
Impacted Area	Cumulative task deficit as % of baseline

Table 2: Eight differential resilience metrics (12 total with sub-metrics). All require a paired baseline.

An Agent Dependency Graph (ADG) — a directed graph where agent i depends on agent j if j blocks i's planned path — estimates cascade depth via BFS after each tick [6].

5. Solvers

Five lifelong solvers are implemented from the literature in Rust (no external solver crates):

Solver	Reference & Characteristic
PIBT	[12] — One-step reactive, $O(n \log n)$
RHCR (3 variants)	[8] — Windowed PBS, PIBT-Window, or Priority A*
Token Passing	[10] — Sequential planning via shared token

Table 3: Lifelong MAPF solvers. One-shot solvers (CBS, LaCAM, LNS2) are excluded; fault resilience requires sustained operation.

6. Demonstration: Fault-Induced Congestion Relief

To demonstrate MAFIS's measurement capabilities, we report a large-scale experiment revealing solver- and topology-dependent behavior undetectable without differential measurement infrastructure.

6.1. Experimental Setup

Configuration. 5 solvers × 4 fleet densities (10, 20, 40, 80) × 7 fault scenarios × 50 seeds = 7,000 paired runs on `warehouse_medium` (32 × 21). Cross-topology validation: 480 additional runs on `sorting_center` and `compact_grid`. Scheduler: random (delivery targets drawn uniformly to elim-

inate positional bias). Tick count: 500. All runs headless with Rayon parallelism (90s total).

Statistical method. Per paired run: Braess ratio = $\overline{\text{throughput}}_{\text{fault}}/\overline{\text{throughput}}_{\text{baseline}}$. Two-sided Mann-Whitney U with Benjamini-Hochberg FDR correction across all 140 tests [3]. Cliff's delta for effect sizes. A finding is *confirmed* when 95% CI lower bound > 1.0 AND BH-adjusted $p < 0.05$.

6.2. Results

Of 140 hypothesis tests, 35 are significant after BH correction; 9 show confirmed congestion relief (ratio > 1, CI > 1, adjusted $p < 0.05$). All 9 occur under burst failure scenarios.

Solver	Threshold	Ratio	Cliff's δ	p_{adj}
PIBT	$n = 10$	1.774 [1.33, 2.22]	0.376	0.006
RHCR-PIBT	$n = 80$	1.608 [1.33, 1.89]	0.347	0.015
RHCR-PBS	none	—	—	—
RHCR-A*	none	—	—	—
Token Passing	none	0.688 [0.64, 0.73]	−0.991	<0.001

Table 4: Braess ratios under burst-20% at the confirmed threshold density. *Threshold* = lowest n where BH-adjusted $p < 0.05$ and CI lower > 1. Token Passing shows no congestion relief at any density. RHCR-PBS and RHCR-A* are never confirmed.

The phenomenon parallels Braess's paradox [4]: removing capacity (killing agents) improves flow for solvers operating in a congested regime. PIBT is locally reactive — each agent optimizes greedily, analogous to selfish routing. The effect appears from $n = 10$ and peaks at $n = 80$ (2.09×). RHCR-PIBT resists until $n = 80$, requiring extreme density before coordination breaks down. Token Passing's sequential token structure is fragile: each death creates a path gap that propagates through the shared constraint set, producing Cliff's $\delta = -0.99$ to -1.00 at low density.

Jiang et al. [7] observed throughput saturation in lifelong MAPF and used intentional agent disabling as an optimization trick. Our result extends this: *unintentional* faults produce the same effect, and the response is architecture-dependent.

6.3. Cross-Topology Validation

The effect is *topology-dependent*. On `compact_grid` (Ocado-style), PIBT at $n = 40$ confirms: ratio 1.431 [1.33, 1.54], $p < 0.001$. On `sorting_center` (3 chokepoints), no density produces a confirmed effect. This indicates the mechanism requires corridor congestion (many alternative routes competing), not structural bottlenecks (few chokepoints limiting throughput). Token Passing collapses on both topologies (8/8 configs below 1.0, all $p < 0.001$).

7. Observatory Features

3D Visualization. Instanced meshes render up to 500 agents in one draw call. Color palettes encode task state (simple: 4 states; detailed: 8). Dual heatmaps (density decay + cumulative traffic) identify hotspots.

Timeline & Rewind. Ring-buffer tick history; scrub to any past tick. Fault events appear as annotated markers.

Experiment Infrastructure. Headless runner produces CSV, JSON, LaTeX, Typst, and SVG outputs. Python analysis script computes BH-corrected significance without external dependencies.

Topologies. Five industry-inspired layouts (Amazon Kiva small/medium/ large, FedEx sorting center, Ocado compact grid) defined as JSON.

8. Related Work

Solver evaluation tools. MAPF Tracker evaluates one-shot solver speed on clean benchmarks. The League of Robot Runners [7] evaluates lifelong throughput under normal conditions. SMART provides kinodynamic fidelity for execution. None inject faults or measure resilience. WareRover couples scheduling and MAPF with a basic fault model (flat probability, binary state, one-shot solvers); MAFIS provides Weibull-calibrated wear, three fault categories, five lifelong solvers, cascade analysis, and browser deployment.

Execution robustness. Hönig et al. [6] introduce ADG for delay-robust execution; Atzmon et al. [2] define k-robust MAPF. Morag et al. [11] handle planning failures in lifelong MAPF. All address delays or planner timeouts — temporary disruptions. MAFIS addresses permanent failures: robot death, zone loss, progressive Weibull wear.

Congestion in MAPF. Chen et al. [5] optimize traffic flow for lifelong MAPF, documenting throughput saturation. Atasoy Bingol et al. [1] formalize "retrograde scalability" in swarm robotics. MAFIS extends these observations to the fault domain: we show that faults can push a congested system past the throughput peak.

Resilience in MAS. Vistbakka & Troubitsyna [15] formalize resilient reconfiguration; Zahradka et al. [16] monitor MAPF execution via ADG with 1,300 experiments on one-shot MAPF. MAFIS operates on lifelong MAPF with 7,480 runs and browser access.

9. Conclusion

MAFIS provides the first browser-based fault resilience observatory for lifelong MAPF. Its three-category taxonomy, twelve differential metrics, and deterministic replay enable studies impossible with existing tools. The demonstration experiment — revealing topology-dependent congestion relief under fault injection, confirmed with BH-FDR correction — shows that

MAFIS can surface non-obvious behaviors warranting targeted solver design.

Limitations. Solvers are reimplemented (not reference code). Maximum tested density is 80 agents; scale validation is future work. The congestion relief effect requires formal modeling to explain the mechanism — planned for a follow-up study.

Live demo: stasis-website.vercel.app/simulator · github.com/stasis-industries/mafis

References

[1] Hazal Atasoy Bingol, Tobias Topfer, Sven Kosub, Heiko Hamann, and Andreagiovanni Reina. 2025. Optimal Scalability-Aware Allocation of Swarm Robots: From Linear to Retrograde Performance via Marginal Gains. *arXiv preprint arXiv:2512.23431* (2025).

[2] Dor Atzmon, Roni Stern, Ariel Felner, Glenn Wagner, Roman Barták, and Neng-Fa Zhou. 2020. Robust Multi-Agent Path Finding and Executing. *Journal of Artificial Intelligence Research* 67, (2020), 549–579. https://doi.org/10.1613/jair.1.11534

[3] Yoav Benjamini and Yosef Hochberg. 1995. Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing. *Journal of the Royal Statistical Society: Series B* 57, 1 (1995), 289–300.

[4] Dietrich Braess. 1968. Über ein Paradoxon aus der Verkehrsplanung. *12*, 258–268.

[5] Zhe Chen, Daniel Harabor, Jiaoyang Li, and Peter J. Stuckey. 2024. Traffic Flow Optimisation for Lifelong Multi-Agent Path Finding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.

[6] Wolfgang Hönig, Scott Kiesel, Andrew Tinka, Joseph W. Durham, and Nora Ayanian. 2019. Persistent and Robust Execution of MAPF Schedules in Warehouses. *IEEE Robotics and Automation Letters* 4, 2 (2019), 1125–1131. https://doi.org/10.1109/LRA.2019.2894217

[7] He Jiang, Yulun Zhang, Oren Salzman, and Jiaoyang Li. 2024. Scaling Lifelong Multi-Agent Path Finding to More Realistic Settings: Research Challenges and Opportunities. In *Proceedings of the International Symposium on Combinatorial Search (SoCS)*, 2024.

[8] Jiaoyang Li, Zhe Chen, Daniel Harabor, Peter J. Stuckey, and Sven Koenig. 2021. Scheduling of Large-Scale Multi-Agent Path Finding via a Grid-Based Decomposition. In *Proceedings of the International Conference on Autonomous Agents and Multi-Agent Systems (AAMAS)*, 2021.

[9] Jiaoyang Li, Andrew Tinka, Scott Kiesel, Joseph W. Durham, T. K. Satish Kumar, and Sven Koenig. 2021. Lifelong Multi-Agent Path Finding in Large-Scale Warehouses. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2021. 11272–11281.

[10] Hang Ma, Jiaoyang Li, T. K. Satish Kumar, and Sven Koenig. 2017. Lifelong Multi-Agent Path Finding for Online Pickup and Delivery Tasks. In *Proceedings of the International Conference on Autonomous Agents and Multi-Agent Systems (AAMAS)*, 2017.

[11] Jonathan Morag, Roni Stern, and Ariel Felner. 2023. Adapting to Planning Failures in Lifelong Multi-Agent Path Finding. In *Proceedings of the International Symposium on Combinatorial Search (SoCS)*, 2023.

[12] Keisuke Okumura and Xavier Defago. 2022. Priority Inheritance with Backtracking for Iterative Multi-agent Path Finding. In *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*, 2022.

[13] Barak Or. 2025. MTTR-A: Measuring Cognitive Recovery Latency in Multi-Agent Systems. *arXiv preprint arXiv:2511.20663* (2025).

[14] Roni Stern, Nathan Sturtevant, Ariel Felner, Sven Koenig, Hang Ma, Thayne Walker, Jiaoyang Li, Dor Atzmon, Liron Cohen, T. K. Satish Kumar, and others. 2019. Multi-Agent Pathfinding: Definitions, Variants, and Benchmarks. In *Proceedings of the International Symposium on Combinatorial Search (SoCS)*, 2019.

[15] Inna Vistbakka and Elena Troubitsyna. 2021. Modelling Resilient Collaborative Multi-Agent Systems. *Computing* 103, (2021), 535–557. https://doi.org/10.1007/s00607-020-00861-2

[16] David Zahradka and others. 2025. A Holistic Architecture for Monitoring and Optimization of Robust MAPF Plan Execution. *arXiv preprint arXiv:2509.10284* (2025).